

INFORME

Evaluación Parcial 1 - Sistema de cuidados de mascotas

Equipo:

Maura Ramirez Navarro
Allison Sepulveda Arenas

Supervisor encargado:

**Valparaíso
Abril, 2026**

ÍNDICE

ÍNDICE.....	2
1. Introducción.....	3
1.1 Planteamiento del problema.....	3
1.3 Objetivo general.....	4
1.5 Justificación.....	4
1.6 Alcance.....	4
3. Diseño de la solución propuesta.....	5
3.1 Arquitectura general.....	5
3.2 Actores del sistema.....	5
3.3 Casos de uso principales.....	6
4. Requerimientos del sistema.....	6
4.1 Requerimientos funcionales.....	6
4.2 Requerimientos no funcionales.....	6
5. Metodología de desarrollo.....	7
5.1 Enfoque de trabajo.....	7
5.2 Etapas del desarrollo.....	7
5.3 Herramientas utilizadas.....	7
5.4 Justificación de la metodología.....	7
6. Diseño visual y usabilidad.....	8
6.1 Paleta de colores.....	8
6.2 Tipografía.....	8
6.3 Pantallas principales.....	8
7. Validaciones y lógica.....	8
7.1 Validaciones.....	8
7.2 Lógica del sistema.....	8
8. Persistencia local y recursos nativos.....	8
8.1 Características.....	9
8.2 Datos almacenados.....	9
8.3 Beneficios.....	9
9. Integración y comunicación.....	9
9.1 Tecnologías.....	9
9.2 Flujo de comunicación.....	9
10. Pruebas y resultados esperados.....	9
11. Desarrollo Backend y Calidad de Software.....	10
Ejemplo de endpoint:.....	10
Pruebas realizadas:.....	11
11.4 Pruebas unitarias.....	11
Ejemplos:.....	11
12. FACTIBILIDAD Y DESPLIEGUE.....	11
12.1 Despliegue del sistema.....	11
12.2 Consideraciones.....	11
13. CONCLUSIÓN.....	12

1. Introducción

1.1 Planteamiento del problema

En la actualidad, el aumento en la tenencia de mascotas ha incrementado la necesidad de llevar un control adecuado sobre su salud y bienestar. Sin embargo, muchos dueños no cuentan con herramientas ni conocimientos suficientes para gestionar información clave como ciclos biológicos, vacunas, controles veterinarios y hábitos de cuidado.

La problemática se origina en la falta de conocimiento y organización, sumado a la ausencia de sistemas centralizados, lo que provoca que esta información se maneje de forma manual, dispersa o dependa de la memoria.

Como consecuencia, se generan olvidos de controles importantes, incumplimiento de calendarios de salud y una detección tardía de problemas, afectando directamente el bienestar de las mascotas y generando preocupación en sus dueños.

1.2 Análisis del problema

Usuarios afectados

- Dueños de mascotas
- Mascotas (impacto indirecto)
- Veterinarios (indirecto)

Causas del problema

- Uso de métodos manuales
- Falta de herramientas digitales especializadas
- Información dispersa
- Ausencia de recordatorios automatizados

Impacto del problema

- Omisión de cuidados (estimado 30-40%)
- Pérdida de información relevante
- 1-2 horas semanales pérdidas en gestión
- Riesgos en la salud de las mascotas

Evidencia

- Observación del comportamiento de usuarios
- Uso frecuente de agendas o memoria
- Falta de soluciones integradas

1.3 Objetivo general

Desarrollar una plataforma digital que permita mejorar la adherencia a las rutinas de cuidado de mascotas, mediante la centralización de información médica, hábitos y recordatorios, con el propósito de reducir omisiones en el cuidado y contribuir al bienestar, la salud y la calidad de vida tanto de las mascotas como de sus dueños.

Con el propósito de reducir omisiones en el cuidado y contribuir al bienestar. Asimismo, el sistema se apoya en APIs externas para enriquecer la información de las mascotas y mejorar la experiencia de uso.

1.4 Objetivos específicos

- Diseñar la arquitectura del sistema basada en microservicios para asegurar escalabilidad y eficiencia.
- Implementar el registro y gestión de usuarios y mascotas, facilitando la administración de la información.
- Desarrollar un módulo de fichas médicas digitales para el almacenamiento y consulta de datos de salud
- Implementar un sistema de recordatorios automatizados para mejorar el cumplimiento de rutinas de cuidado.
- Desarrollar funcionalidades para el seguimiento de hábitos, contribuyendo al control del bienestar de las mascotas.
- Garantizar la seguridad y persistencia de los datos, resguardando la información de los usuarios
- Integrar datos externos desde TheDogAPI y TheCatAPI, permitiendo consultar razas, características y recursos visuales de forma estandarizada mediante servicios REST.

1.5 Justificación

El proyecto responde a la necesidad de mejorar la organización y control del cuidado de mascotas, reduciendo riesgos asociados a la omisión de actividades importantes como controles veterinarios, vacunación y seguimiento de hábitos.

Además, permite aplicar conocimientos de desarrollo de software, arquitectura de sistemas y diseño de soluciones tecnológicas centradas en el usuario.

Su implementación generará un impacto positivo al facilitar la gestión del cuidado de las mascotas, contribuyendo a una mejor toma de decisiones por parte de los dueños, disminuyendo errores por desorganización y promoviendo el bienestar, la salud y la calidad de vida tanto de las mascotas como de sus dueños.

Adicionalmente, el sistema incorpora información proveniente de fuentes externas mediante las APIs públicas **TheDogAPI** y **TheCatAPI**, las cuales permiten complementar dinámicamente los datos relacionados con razas, características morfológicas y bancos de imágenes de mascotas. Esta integración mejora la experiencia del usuario al proporcionar información validada y actualizada sin necesidad de mantener estos datos de forma local, optimizando así el rendimiento y reduciendo costos de mantenimiento del sistema.

1.6 Alcance

El proyecto contempla el desarrollo de una plataforma web funcional que incluye:

- Gestión de usuarios
- Gestión de mascotas
- Registro de fichas médicas
- Sistema de recordatorios
- Seguimiento de hábitos

Exclusiones

- No incluye aplicación móvil en fase inicial
- No incluye integración con veterinarias externas
- No incluye sistema de pagos

1.7 Limitaciones del Sistema

El sistema presenta las siguientes limitaciones en su fase inicial:

- Dependencia de conexión a internet para su funcionamiento.
- No contempla integración con clínicas veterinarias externas.
- No incluye aplicación móvil en esta etapa del proyecto.
- Las notificaciones no se implementan en tiempo real (push), sino mediante recordatorios programados.

2. Marco teórico

El desarrollo **del sistema** se basa en los siguientes conceptos fundamentales del área de ingeniería de software:

- Arquitectura de microservicios: enfoque que permite dividir el sistema en servicios independientes, facilitando la escalabilidad y mantenibilidad.
- ● API REST: estilo arquitectónico para la comunicación entre sistemas mediante HTTP.
- ● Normalización de bases de datos: proceso para evitar redundancia y asegurar integridad.
- ● Seguridad de la información: aplicación de mecanismos de autenticación, autorización y cifrado.

2.1 Stakeholders

Los principales interesados del sistema son:

Stakeholders	Descripción
Dueños de mascotas	Usuarios finales del sistema
Mascotas	Beneficiarios indirectos del sistema
Veterinarios	Posibles usuarios futuros del sistema
Equipo de desarrollo	Encargados del diseño e implementación
Institución académica	Entidad evaluadora del proyecto

2.2 Análisis de riesgos

El desarrollo del sistema puede enfrentar los siguientes riesgos:

Riesgo	Impacto	Mitigación
Fallo del sistema	Pérdida de información	Implementación de backups
Baja adopción de usuarios	Bajo uso del sistema	Mejora de UX/UI
Errores en desarrollo	Fallos en funcionalidades	Pruebas constantes
Problemas de seguridad	Exposición de datos	Implementación de cifrado

3. Diseño de la solución propuesta

3.1 Arquitectura general

El sistema se basa en una arquitectura de microservicios compuesta por:

- Frontend web
- API Gateway
- Microservicios independientes:
 1. Usuarios
 2. Mascotas
 3. Salud
 4. Recordatorios
 5. Hábitos

Cada microservicio cuenta con su propia base de datos.

La arquitectura considera además un módulo de integración externa encargado de consumir de forma segura y controlada los endpoints de TheDogAPI y TheCatAPI. Este componente actúa como un *microservicio cliente* responsable del manejo de solicitudes HTTP salientes, administración de claves de API, estandarización de respuestas y desacoplamiento del frontend respecto de las APIs externas

3.1.1 Descripción de Microservicios

El sistema se compone de cinco microservicios independientes, diseñados bajo el patrón *Database per Service* y comunicados mediante API REST a través del API Gateway. Cada servicio encapsula un dominio específico:

- **Microservicio de Usuarios:** Gestiona el registro, autenticación y administración de cuentas de usuario.
- **Microservicio de Mascotas:** Administra la creación, edición, consulta y eliminación de mascotas asociadas a cada usuario.
- **Microservicio de Salud:** Registra eventos médicos como vacunas, tratamientos y controles veterinarios.
- **Microservicio de Recordatorios:** Permite programar recordatorios asociados a acciones o eventos del cuidado de la mascota.
- **Microservicio de Hábitos:** Registra actividades rutinarias y permite generar métricas simples como rachas de cumplimiento.

Cada microservicio mantiene su propia base de datos en MySQL, garantizando un desacoplamiento completo, escalabilidad y mayor mantenibilidad del sistema.

3.2 Actores del sistema

La elección de una arquitectura basada en microservicios responde a la necesidad de construir un sistema escalable, desacoplado y mantenible. Cada servicio se encarga de una funcionalidad específica, permitiendo su desarrollo, despliegue y escalamiento independiente.

3.3 Diagrama de Contexto

El Diagrama de Contexto (Nivel 0) representa al *Sistema de Cuidados de Mascotas* como una única entidad lógica y muestra las interacciones principales entre los actores externos y el sistema. Este diagrama permite comprender de manera global el alcance y las fronteras del sistema sin detallar aún su funcionamiento interno.

(FALTA)

3.3 Actores del Sistema

Actores	Descripción
Usuario	Dueño de mascota que utiliza el sistema.
Sistema	Plataforma que gestiona datos y notificaciones.

3.4 Casos de uso principales

Nombre del caso de uso	Actor	Descripción
Registrar mascota	Usuario	El usuario ingresa al sistema y registra una nueva mascota proporcionando sus datos básicos, permitiendo su almacenamiento y posterior gestión dentro de la plataforma
Registrar evento médico	Usuario	El usuario selecciona una mascota y registra un evento médico, como una vacuna o tratamiento, quedando almacenado para su seguimiento y consulta futura.
Crear recordatorio	Usuario	El usuario configura un recordatorio asociado a una mascota para una actividad específica, el cual será notificado en la fecha programada.
Registrar hábito	Usuario	El usuario ingresa información sobre actividades diarias de la mascota, permitiendo llevar un seguimiento continuo de sus rutinas.
Consultar historial	Usuario	El usuario accede al perfil de una mascota y visualiza su historial completo, incluyendo eventos médicos, hábitos y recordatorios registrados.

3.5 Diagrama de Componentes

El sistema se estructura bajo una arquitectura basada en componentes, organizada en capas, lo que permite separar responsabilidades, mejorar la mantenibilidad y facilitar la escalabilidad del sistema.

La **capa de presentación** corresponde al frontend desarrollado en React, encargado de gestionar la interacción con el usuario y la visualización de la información.

La **capa de acceso** está representada por un API Gateway, el cual centraliza y gestiona las solicitudes provenientes del cliente, actuando como punto de entrada hacia los distintos servicios del sistema.

El backend se organiza en una arquitectura por capas compuesta por:

- **Controladores:** responsables de gestionar las solicitudes HTTP entrantes y devolver respuestas al cliente.
- **Servicios:** contienen la lógica de negocio, procesando la información y aplicando las reglas del sistema.
- **Repositorios:** encargados de la interacción directa con la base de datos, permitiendo operaciones de persistencia de datos.

Finalmente, la **capa de persistencia** está compuesta por bases de datos independientes, implementadas en MySQL, lo que permite garantizar el desacoplamiento, la escalabilidad y la integridad de la información.

FALTA

3.6 Modelo de Datos (ERD)

El modelo de datos del sistema se estructura mediante un diagrama entidad-relación que representa las principales entidades del dominio y sus relaciones. El diseño se fundamenta en la normalización y en la separación por microservicio, aplicando el patrón *Database per Service*.

Las relaciones clave son:

- Un **usuario** posee una o varias **mascotas** (1:N).
- Una **mascota** puede tener múltiples **eventos médicos** (1:N).
- Una **mascota** puede registrar diversos **hábitos** (1:N).
- Una **mascota** puede tener uno o varios **recordatorios** (1:N).

Este modelo permite mantener integridad referencial, facilitar consultas y asegurar que la información crítica del dominio se encuentre correctamente estructurada.

3.7 Diagrama de Clase

El modelo de dominio representa las entidades principales del sistema desde una perspectiva orientada a objetos. Incluye atributos esenciales y las relaciones entre clases, proporcionando una visión conceptual de la estructura interna de la aplicación.

(falta)

3.8 Diagrama de Secuencia – Registrar Mascota

Este diagrama describe el flujo de interacción entre el usuario, la interfaz web, el API Gateway y el microservicio de mascotas durante el registro de una nueva mascota.

Pasos principales:

1. El usuario ingresa los datos de la mascota.
2. El frontend envía la solicitud al API Gateway.
3. El API Gateway delega la operación al microservicio de mascotas.
4. El microservicio valida los datos y los guarda en la base de datos.
5. Se retorna una respuesta de éxito al usuario.

3.9 Diagrama de Secuencia – Crear Recordatorio

La siguiente secuencia describe el proceso de creación de un recordatorio asociado a una mascota:

1. El usuario selecciona una mascota y solicita crear un recordatorio.
2. El frontend envía la solicitud al API Gateway.
3. El Gateway redirige la petición al microservicio de recordatorios.
4. El microservicio valida la fecha y la categoría.
5. Se guarda el recordatorio en la base de datos.
6. El sistema confirma la creación.

4. Requerimientos del sistema

4.1 Requerimientos funcionales

ID	Requisito Funcional	Descripción
RF01	Registro de usuarios	El sistema debe permitir el registro de usuarios mediante correo electrónico y contraseña.
RF02	Autenticación de usuarios	El sistema debe permitir la autenticación de usuarios en la plataforma.
RF03	Registro de mascotas	El sistema debe permitir registrar mascotas asociadas a un usuario.
RF04	Gestión de mascotas	El sistema debe permitir editar y eliminar mascotas registradas.
RF05	Registro de eventos médicos	El sistema debe permitir registrar eventos médicos, tales como vacunas y tratamientos.
RF06	Recordatorios automatizados	El sistema debe permitir crear recordatorios automáticos para eventos importantes.

RF07	Seguimiento de hábitos	El sistema debe permitir registrar hábitos diarios de la mascota.
RF08	Historial de mascota	El sistema debe mostrar el historial completo de cada mascota.
RF09	Notificaciones	El sistema debe generar notificaciones al usuario.
RF10	Consumo de API externa	El sistema debe integrar información de razas e imágenes provenientes de TheDogAPI y TheCatAPI, permitiendo su uso dentro del módulo de mascotas.

4.1.1 Reglas de Negocio

Las reglas de negocio que rigen el funcionamiento del sistema son las siguientes:

- **RN01:** Un usuario solo puede visualizar y administrar la información de sus propias mascotas.
- **RN02:** Toda mascota debe pertenecer obligatoriamente a un usuario registrado.
- **RN03:** No se permiten eventos médicos cuya fecha exceda los 30 días futuros desde la fecha actual.
- **RN04:** Para crear un recordatorio, se debe especificar al menos una fecha y una categoría de actividad.
- **RN05:** Una mascota no puede eliminarse si posee eventos médicos registrados en su historial.

Estas reglas permiten mantener la consistencia y validez de los datos dentro del sistema.

4.2 Requerimientos no funcionales

ID	Requisito Funcional	No	Descripción
RNF01	Rendimiento		El sistema debe responder a las solicitudes en un tiempo máximo de 3 segundos bajo condiciones normales de operación.
RNF02	Seguridad		El sistema debe garantizar la protección de datos mediante autenticación basada en JWT, cifrado de contraseñas y uso de HTTPS.
RNF03	Disponibilidad		El sistema deberá garantizar una disponibilidad mínima del 99% anual.
RNF04	Usabilidad		El sistema debe ofrecer una interfaz intuitiva, consistente y fácil de aprender, permitiendo que usuarios con distintos niveles de experiencia puedan interactuar eficientemente con sus funcionalidades.
RNF05	Escalabilidad		El sistema debe ser intuitivo, fácil de usar y accesible para todo tipo de usuarios.

RNF06	Compatibilidad	El sistema debe ser capaz de incrementar su capacidad de procesamiento, almacenamiento y manejo de carga sin afectar su rendimiento, permitiendo el crecimiento en cantidad de usuarios, datos y microservicios.
--------------	-----------------------	--

5. Metodología de desarrollo

El proyecto será desarrollado en equipo utilizando la metodología ágil Scrum, la cual permitirá organizar el trabajo de manera iterativa, flexible y orientada a la mejora continua.

Este enfoque facilitará la adaptación a cambios durante el desarrollo, así como una gestión eficiente de las tareas y del tiempo, promoviendo la colaboración entre los integrantes del equipo.

5.1 Enfoque de trabajo

El proyecto se organizará mediante iteraciones cortas denominadas **sprints**, en las cuales se planificará, desarrollará y evaluará distintas funcionalidades del sistema.

Cada sprint permitirá avanzar progresivamente desde la comprensión del problema hasta la implementación de la solución, asegurando una evolución continua del proyecto.

Asimismo, se mantendrá una organización estructurada del trabajo, permitiendo validar cada etapa antes de avanzar a la siguiente, garantizando coherencia entre los requerimientos, objetivos y resultados esperados.

5.2 Etapas del desarrollo

El desarrollo del proyecto se organizará en las siguientes etapas:

- **Análisis del problema**

Identificación de la problemática, definición de los usuarios afectados, análisis de causas y evaluación del impacto en el contexto definido.

- **Definición del objetivo**

Formulación del objetivo general del proyecto, considerando criterios claros y medibles.

- **Definición del alcance**

Determinación de los límites del sistema, incluyendo funcionalidades principales, restricciones y exclusiones.

- **Diseño de la solución**

Definición de la arquitectura del sistema, estructura de componentes, diseño de base de datos y elaboración de diagramas.

- **Desarrollo**

Implementación de las funcionalidades del sistema e integración de servicios necesarios.

- **Validación**

Revisión de la coherencia entre el problema, los objetivos y la solución propuesta.

5.3 Herramientas utilizadas

- GitHub: control de versiones del proyecto
- Herramientas de documentación (Word, Drive)
- Apoyo en herramientas de modelado (diagramas)

5.4 Justificación de la metodología

El uso de un enfoque ágil adaptado permitió mantener flexibilidad durante el desarrollo, facilitando la mejora continua del proyecto y la detección y corrección de errores en etapas tempranas.

Además, este enfoque resulta especialmente adecuado para proyectos grupales, ya que permite una adecuada coordinación entre los integrantes del equipo, favoreciendo la distribución de tareas, la comunicación constante y el seguimiento del avance de cada fase. De esta manera, se optimiza la gestión del tiempo y se asegura el cumplimiento de los objetivos del proyecto.

5.5 Planificación del Proyecto

El desarrollo del proyecto se estructura en fases secuenciales, siguiendo un enfoque organizado que permite controlar el avance, asegurar la calidad del producto y cumplir con los objetivos definidos. Cada fase contempla actividades específicas y entregables asociados.

Cada fase será desarrollada bajo una planificación temporal definida (Gantt), permitiendo visualizar la duración de las actividades, dependencias y control del progreso del proyecto.

A continuación, se detallan las etapas del proyecto:

Fase	Nombre de la Fase	Descripción de Actividades
Fase 1	Análisis del problema	Identificación de necesidades, levantamiento de requerimientos, análisis del contexto y definición del alcance del sistema.
Fase 2	Diseño de la solución	Elaboración de la arquitectura del sistema, diseño de base de datos, diagramas (ERD, componentes) y definición de tecnologías a utilizar

Fase 3	Desarrollo	Implementación del sistema, construcción de funcionalidades, integración de servicios externos (servicios de notificación y gestión de eventos, etc.).
Fase 4	Pruebas	Ejecución de pruebas funcionales, detección y corrección de errores, validación del sistema respecto a los requerimientos definidos.
Fase 5	Documentación	Elaboración de informes técnicos, manuales de usuario y documentación del sistema para su mantenimiento y escalabilidad.

6. Diseño visual y usabilidad

6.1 Paleta de colores

Se propone una paleta de colores amigable, moderna y asociada al cuidado de mascotas, buscando generar cercanía y confianza en el usuario.

- Verde menta = #6FCF97 para botones principales, acciones positivas.
- Naranja suave = #F2994A Alertas, recordatorios.
- Celeste claro = #56CCF2 Fondos secundarios.
- Blanco = #FFFFFF Fondo principal.
- Gris claro = #F2F2F2 Contenedores.
- Gris oscuro = #4F4F4F Texto

6.2 Tipografía

Se utilizó la tipografía Poppins para los títulos, subtítulos y textos.

6.3 Prototipo Visual – Interfaz en Modo Claro

La siguiente figura presenta la propuesta visual del modo claro, donde se aprecia la aplicación práctica de la paleta de colores definida.



6.4 Prototipo Visual – Interfaz en Modo Oscuro

En la figura se muestra la adaptación de la paleta al modo oscuro, optimizada para entornos de baja luminosidad.



6.5 Pantallas principales

- Dashboard
- Registro de mascotas
- Historial médico
- Recordatorios
- Seguimientos de hábitos

7. Validaciones y lógica

7.1 Validaciones

- Campos obligatorios en formularios
- Validación de formato (texto, fechas)
- Validación de datos duplicados
- Validación de fechas futuras en recordatorios
- Validación de consistencia en eventos médicos
- Validación de autenticación mediante tokens JWT.
- Validación de permisos según rol de usuario.

7.2 Lógica del sistema

- Asociación de mascotas a usuarios
- Relación entre eventos médicos y mascotas
- Generación automática de recordatorios
- Cálculo de hábitos y rachas
- Actualización dinámica del historial

8. Persistencia local y recursos nativos

El sistema utilizará una base de datos estructurada para almacenar la información de forma segura y organizada.

8.1 Características

- Base de datos independiente por microservicio
- Almacenamiento estructurado
- Acceso mediante API

8.2 Datos almacenados

- Usuarios
- Mascotas
- Eventos médicos
- Hábitos
- Recordatorios

8.3 Beneficios

- Seguridad de la información
- Escalabilidad
- Mantenibilidad

9. Integración y comunicación

9.1 Tecnologías

El sistema se propone utilizando las siguientes tecnologías:

- **Frontend:** React.js
- **Backend:** Node.js con Express
- **Base de datos:** MySQL
- **Comunicación:** API REST (HTTP/JSON)
- **Control de versiones:** GitHub
- **Herramientas de prueba:** Postman

9.2 Flujo de comunicación

Usuario → Frontend → API Gateway → Microservicios → Base de datos

9.3 Integración con APIs externas

El sistema incorpora integración con las APIs públicas TheDogAPI y TheCatAPI, las cuales proveen información estructurada sobre razas, imágenes, categorías y características de mascotas.

Esta integración se implementa desde un microservicio especializado que realiza las solicitudes HTTP hacia los endpoints externos, procesa las respuestas y las expone al frontend mediante una interfaz REST interna.

Propósito de la integración

- Acceder a catálogos actualizados de razas sin mantener una base de datos propia.
- Obtener imágenes de alta calidad para mejorar la experiencia visual del usuario.
- Reducir la carga de mantenimiento del sistema delegando datos estáticos en servicios externos.
- Asegurar consistencia y estandarización en los datos utilizados por el módulo de mascotas.

Endpoints consumidos

TheDogAPI

- /v1/breeds → Catálogo de razas
- /v1/images/search → Imágenes por raza

TheCatAPI

- /v1/breeds → Catálogo de razas felinas
- /v1/images/search → Imágenes por categoría

Proceso de integración

1. El frontend solicita información de razas al sistema.
2. El API Gateway redirige la petición al microservicio de integración externa.
3. El microservicio consume la API externa mediante solicitudes HTTPS.
4. La información recibida se normaliza en formato JSON interno.
5. Se retorna la respuesta al frontend a través del API Gateway.

Beneficios para la experiencia de usuario

- Información de razas disponible en tiempo real y sin duplicación.
- Imágenes dinámicas para mejorar interfaces de perfil de mascotas.
- Optimización de recursos al no almacenar grandes bibliotecas de imágenes.
- Mantenimiento simplificado al depender de APIs especializadas en datos de mascotas.

10. Pruebas y resultados esperados

Con el objetivo de garantizar la calidad, estabilidad y correcto funcionamiento del sistema, se definen distintos tipos de pruebas que permiten validar tanto la lógica como el comportamiento de la aplicación.

El plan de pruebas se enfoca en validar las funcionalidades principales del sistema, asegurando su correcto funcionamiento y coherencia con los requerimientos definidos. Las pruebas consideran aspectos funcionales, integración entre frontend–backend, y verificación de los microservicios mediante Postman.

Los casos de prueba principales son:

- **PT01 – Registro de mascota:** Con datos válidos la mascota debe almacenarse correctamente.
- **PT02 – Registro fallido de mascota:** Si el nombre está vacío, el sistema debe mostrar un mensaje de validación.
- **PT03 – Crear recordatorio:** Debe almacenarse en la base de datos y aparecer en el listado correspondiente.
- **PT04 – Inicio de sesión:** Debe aceptar credenciales válidas y rechazar credenciales inválidas.

10.1 Tipos de pruebas

Se consideran los siguientes tipos de pruebas:

- **Pruebas funcionales:**
Permiten verificar que cada funcionalidad del sistema cumple con los requerimientos definidos.
- **Pruebas de integración:**
Evalúan la correcta comunicación entre los distintos componentes del sistema (frontend, backend y base de datos).
- **Pruebas de backend:**
Validan el correcto funcionamiento de los endpoints de la API REST, asegurando respuestas adecuadas ante diferentes solicitudes.
- **Pruebas unitarias:**
Se enfocan en validar componentes individuales del sistema, como funciones o métodos específicos.

10.2 Herramientas de prueba

Para la ejecución de pruebas se utilizan las siguientes herramientas:

- **Postman:** para pruebas de endpoints de la API
- **Navegador web:** para pruebas de interfaz de usuario
- **Consola de desarrollo:** para detección de errores

10.3 Casos de prueba

A continuación, se presentan algunos casos de prueba definidos:

ID	Prueba	Resultado esperado
PT01	Registro de mascota con datos válidos	El sistema almacena la información en la base de datos y muestra mensaje de éxito
PT02	Registrar mascota con datos incompletos	El sistema bloquea el registro y muestra mensaje de validación
PT03	Registrar evento médico	El evento se guarda correctamente en el historial de la mascota
PT04	Crear recordatorio	El sistema agenda correctamente la notificación
PT05	Consultar historial	Se visualiza correctamente la información almacenada

PT06	Inicio de sesión	Acceso permitido con datos correctos y rechazado con datos incorrectos
------	------------------	--

10.4 Resultados esperados

Se espera que el sistema:

- Responda correctamente a todas las solicitudes
- No presente errores críticos
- Mantenga consistencia en los datos
- Permita una experiencia fluida al usuario

11. Desarrollo Backend y Calidad de Software

El backend incorpora un microservicio adicional denominado *External Data Service*, encargado de gestionar la comunicación con **TheDogAPI** y **TheCatAPI**. Este servicio utiliza peticiones HTTP tipo GET, manejo de API Keys, validación de respuestas y estandarización de datos para asegurar compatibilidad con el resto de los microservicios. Su inclusión permite desacoplar la lógica de consumo de APIs externas, aumentar la mantenibilidad y centralizar el control de dependencias externas en un único punto.

11.1 Creación y Conexión del Backend

Se propone el desarrollo de una API REST para gestionar la lógica del sistema, actuando como intermediario entre la base de datos y la interfaz de usuario.

El backend se estructura bajo una arquitectura en capas:

- Controladores: gestionan las solicitudes del cliente
- Servicios: contienen la lógica de negocio
- Repositorios: gestionan el acceso a datos

Esta arquitectura permite un sistema escalable, mantenible y organizado.

11.2 Microservicio Local y Consumo de API

El sistema se basa en una arquitectura de microservicios, donde cada componente cumple una función específica:

- Microservicio de usuarios
- Microservicio de mascotas
- Microservicio de salud
- Microservicio de recordatorios
- Microservicio de hábitos

La comunicación entre el frontend y backend se realiza mediante API REST, utilizando formato JSON.

Ejemplo de endpoint:

POST /api/mascotas

Función: Registrar una nueva mascota en el sistema.

11.3 Pruebas del backend

Para validar el correcto funcionamiento del sistema, se realizan pruebas mediante herramientas de simulación de API.

Pruebas realizadas:

- Verificación de endpoints
- Validación de respuestas del servidor
- Pruebas de inserción de datos
- Pruebas de errores (datos inválidos)

Estas pruebas permiten asegurar que el sistema responde correctamente ante distintos escenarios.

11.4 Pruebas unitarias

Se plantean pruebas unitarias para validar la lógica del sistema de forma independiente.

Ejemplos:

- Validación de datos ingresados
- Verificación de creación de registros
- Validación de lógica de recordatorios

El objetivo es garantizar la estabilidad y correcto funcionamiento del sistema.

11.5 Seguridad del Sistema

Para garantizar la protección de los datos, el sistema implementa:

- Autenticación mediante JWT (JSON Web Tokens)
- Encriptación de contraseñas utilizando bcrypt
- Uso de HTTPS para comunicación segura
- Control de acceso basado en roles
- Validación de datos en frontend y backend

12. FACTIBILIDAD Y DESPLIEGUE

12.1 Despliegue del sistema

El sistema será desplegado en un entorno web, permitiendo el acceso desde distintos dispositivos, tales como computadores, tablets y teléfonos móviles, a través de un navegador. Esto facilita su disponibilidad y uso por parte de los usuarios en cualquier momento y lugar.

12.2 Consideraciones

Para el correcto funcionamiento del sistema, se deben tener en cuenta las siguientes consideraciones:

- Acceso a internet para la utilización de la plataforma.
- Implementación de protocolos de seguridad en la comunicación, como HTTPS.
- Capacidad de escalabilidad del sistema para soportar un aumento en la cantidad de usuarios y datos.

12.3 Factibilidad

Factibilidad técnica:

El proyecto es técnicamente viable, ya que puede ser desarrollado utilizando tecnologías actuales y ampliamente utilizadas, como API REST, bases de datos relacionales y una arquitectura basada en microservicios, lo que permite un desarrollo modular, escalable y mantenible.

Factibilidad económica:

El desarrollo presenta un bajo costo de implementación, debido al uso de herramientas y tecnologías de código abierto, como frameworks de desarrollo y plataformas de control de versiones como GitHub, lo que reduce significativamente la inversión requerida.

Factibilidad operativa:

El sistema está diseñado con un enfoque centrado en el usuario, ofreciendo una interfaz intuitiva y de fácil uso, lo que permite su adopción por parte de personas sin conocimientos técnicos avanzados, facilitando su implementación y uso en la vida cotidiana.

12.4 Trabajos Futuros

El proyecto presenta diversas oportunidades de mejora e integración futura, tales como:

- Desarrollo de una aplicación móvil para ampliar la accesibilidad.
- Integración con clínicas veterinarias para gestión profesional de datos.
- Implementación de notificaciones en tiempo real.
- Incorporación de inteligencia artificial para recomendaciones de cuidado.

13. CONCLUSIÓN

El proyecto desarrollado aborda de manera efectiva la problemática asociada a la gestión del cuidado de mascotas, mediante la propuesta de una solución tecnológica basada en una arquitectura de microservicios, que permite una mayor escalabilidad, organización y eficiencia en el manejo de la información.

La plataforma desarrollada facilita la centralización de datos relevantes, tales como información médica, hábitos y recordatorios, permitiendo a los usuarios mejorar la organización y el seguimiento de las rutinas de cuidado. Esto contribuye directamente a la reducción de omisiones y a una mejor toma de decisiones, generando un impacto positivo en la salud, bienestar y calidad de vida tanto de las mascotas como de sus dueños.

Asimismo, el sistema presenta una alta proyección de crecimiento, permitiendo la incorporación de nuevas funcionalidades e integraciones futuras, adaptándose a las necesidades cambiantes de los usuarios. En este sentido, el proyecto se consolida como una solución viable, innovadora y alineada con las tendencias actuales del desarrollo de software y las demandas del entorno digital.

El sistema propuesto no solo permite el registro de información, sino que incorpora un enfoque integral basado en el seguimiento de hábitos y la prevención, diferenciándose de soluciones tradicionales que se enfocan únicamente en el almacenamiento de datos.